FINM 32950: Intro to HPC in Finance

Lecture 1

Chanaka Liyanarachchi

March 25, 2026

Teaching Team

- Instructor: Chanaka Liyanarachchi
  chanaka@uchicago.edu

- TAs:
  Alex Kay (akay@uchicago.edu)

Introduction

Software Optimization: Structured Approach

Appendix
- Appendix A
- Appendix B

Introduction

- This course focuses on performance-oriented computing.
- Our goal is clear: efficiently utilize available resources to develop high-performance software.
- Performance is critically important in Finance:
  - Many application areas are highly sensitive to computational efficiency
  - Examples include: model calibration, risk management, portfolio management, machine learning, and trading
- High-Performance Computing (HPC) can have a direct and significant impact on these important applications.

Topics

1. Structured approach to software optimization.
2. Parallel computing using accelerators:
   - Achieving high performance using a combination of specialized hardware and software solutions
   - There are many popular solutions (no single *standard* technology for all HPC needs):
     - Each solution offers different features
     - Utilizes different hardware resources
     - Has its own advantages and limitations
     - Multiple technologies can be combined within a single application
   - We will look at several popular technologies and programming models.
   - Use a *pattern based* approach to introduce parallalism.

Software Optimization: Structured Approach

Profile-Guided Optimization

- We begin by introducing profile-guided optimization.
- Using an example that prices 10 million (distinct) European call options using the Black-Scholes formula.
- Our primary performance metric is speed.
- This example will be used to illustrate:
  - **Profile-guided optimization**: a structured approach to improving performance using a *Profiler*.
  - How performance limitations motivate the need for parallel programming

Intel Toolkit

- We use various tools and toolkits to develop performance-sensitive applications.
- We begin by exploring the Intel Toolkit, which provides a range of features for building HPC applications:
  - Optimized libraries
  - Language extensions
  - Development tools: `https://www.intel.com/content/www/us/en/developer/tools/oneapi/components.html#gs.ykrlmf`
    1. VTune Profiler: used to guide optimization (profile-guided optimization)
    2. Advisor: used to design and analyze code for efficient vectorization

Example: Black Scholes Pricer

- We will use the code below for this exercise.
- call_price() takes arrays of each option attribute (strike and time-to-expiration) and the parameters (stock price, volatility, and rate) needed to price a call option using Black Scholes.

```
void call_price(float* K, float* T, float* S, float* v,
            float* r, float* C, int NumOptions)
{
  for (int i = 0; i < NumOptions; ++i)
  {
    float d1 =
     (log(S[i] / K[i]) +
     (r[i] + 0.5*v[i] * v[i]) * T[i])/(v[i] * sqrt(T[i]));
    float d2 =
     (log(S[i] / K[i]) +
     (r[i] - 0.5*v[i] * v[i]) * T[i])/(v[i] * sqrt(T[i]));

    float nd1 = cdf_normal(d1);
    float nd2 = cdf_normal(d2);

    C[i] = S[i] * nd1 - K[i] * exp(-r[i] * T[i])*nd2;
  }
}
```

Time Measurements

- We measure the time required to price 10 million (distinct) call options.
- Such time measurements are essential for performance optimization:
  - The initial measurement is called the *baseline*.
  - All subsequent measurements are compared against the baseline.
  - This helps us evaluate the impact of changes (before vs. after optimization).
- However, time measurements alone do not reveal where performance issues exist.
- Question: How do we identify performance bottlenecks and decide what to optimize?

80-20 Rule

- The overall performance of an application is often determined by a small portion of the code.
- This is commonly known as the 80-20 rule:
  - Roughly: *80% of execution time is spent in 20% of the code*
- To improve performance, we must identify these *performance sensitive* sections.

Hotspots

- Sections of code where a program spends most of its execution time are called *hotspots*.
- Effective optimization requires a systematic way to identify these hotspots.
- Profilers are tools designed to locate and analyze hotspots in a program.

- Detour: Using Midway (Appendix A)

Appendix A

Getting Demo Code and Loading Software

- I post weekly demo code under: /project2/finm32950/chanaka/
- For this demo you will use L1Demo.tar (on midway3: /project2/finm32950/chanaka/L1Demo.tar).
- Copy demo code to your home directory:
  `cp /project2/finm32950/chanaka/L1Demo.tar .`
- Uncompress:
  `tar -xvf L1Demo.tar`
- List
  `ls -F`
- Change directory to L1Demo:
  `cd L1Demo/Profiling`

  `ls`
- bs1.cpp is our program.

Building the Program

- Use Intel C++ compiler (icc) to build the program.
- On midway3, we first need to load Intel module (version 2022.0)

```
module load intel/2022.0
```

- To build using icc:

```
icc -std=c++11 -o bs1 bs1.cpp
```

- For help:

```
icc -help
```

Makefiles

- A more scalable approach to building programs is to use a Makefile (provided).
- Makefiles are especially useful for non-trivial builds involving:
  - Multiple source files
  - External libraries
  - Complex dependencies
- `make` is a build automation tool that uses a `Makefile`.
- To build the project:

  ```
  make
  ```

- Resources:
  - Tutorial: `https://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/`
  - Reference:
    1. `https://www.gnu.org/software/make/manual/make.html`
    2. Robert Mecklenburg (2004). *Managing Projects with GNU Make*. O'Reilly.

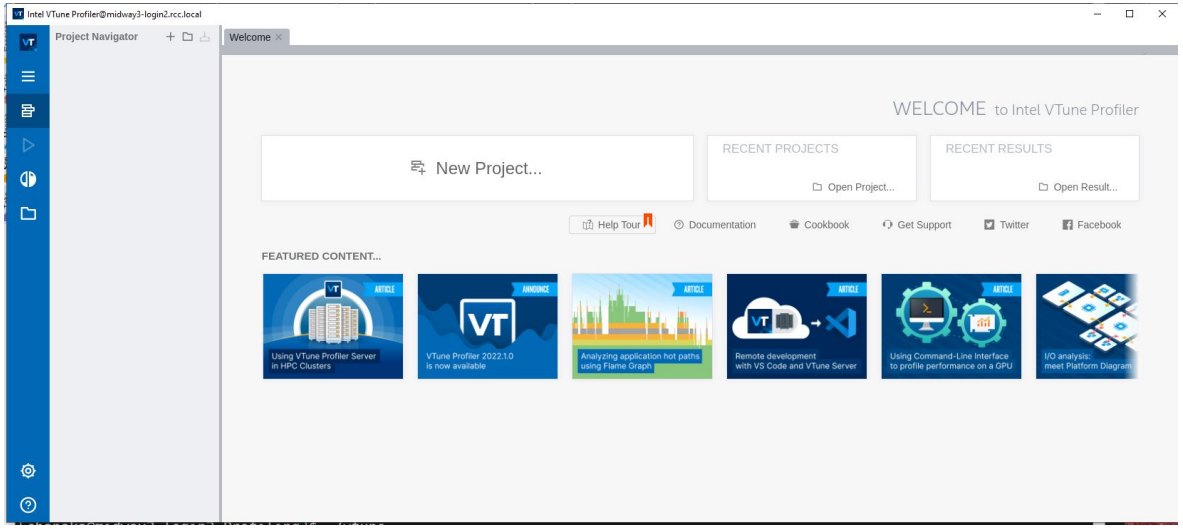
CMake

- CMake is a cross-platform build system generator.
- It allows you to define builds using a simple `CMakeLists.txt` file.
- Works well with Intel compilers and large-scale projects.
- Minimal example provided
- Build steps:
  1. Create a separate build directory:
     ```
     mkdir build
     cd build
     ```
  2. Generate build files:
     ```
     cmake ..
     ```
  3. Compile the project:
     ```
     make
     ```
  4. The executable will be located in the `build` directory.

Profiling Using VTune

Launching the profiler:

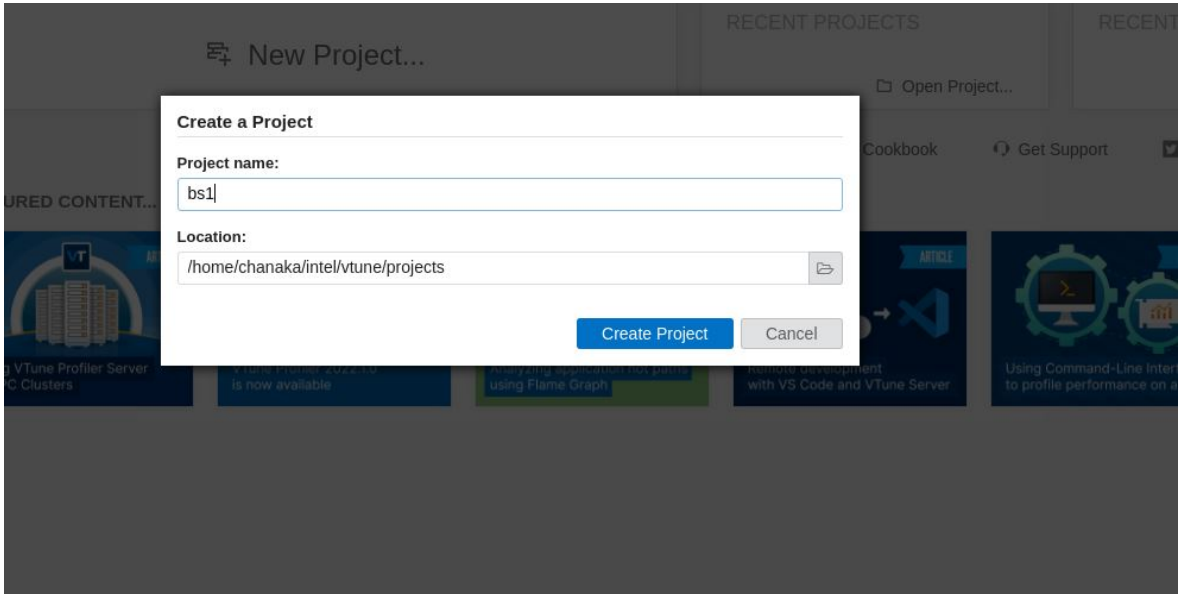
- Navigate to the profiling directory:
  `cd L1Demo/Profiling`
- Run the VTune launcher script:[1]
  `./run_vtune`
- This will open the VTune Profiler interface:



[1] This script is a custom helper utility to simplify launching VTune

Creating a VTune Project

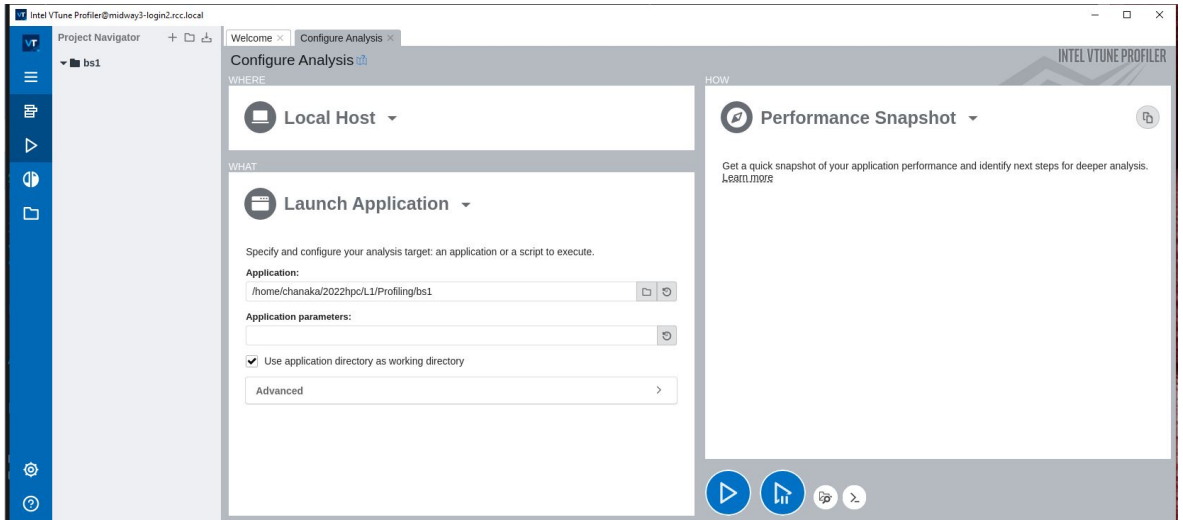
- Select *New Project* to create a new profiling project.
- Provide a name for the project (any name is fine).



- Click *Create Project* to proceed.

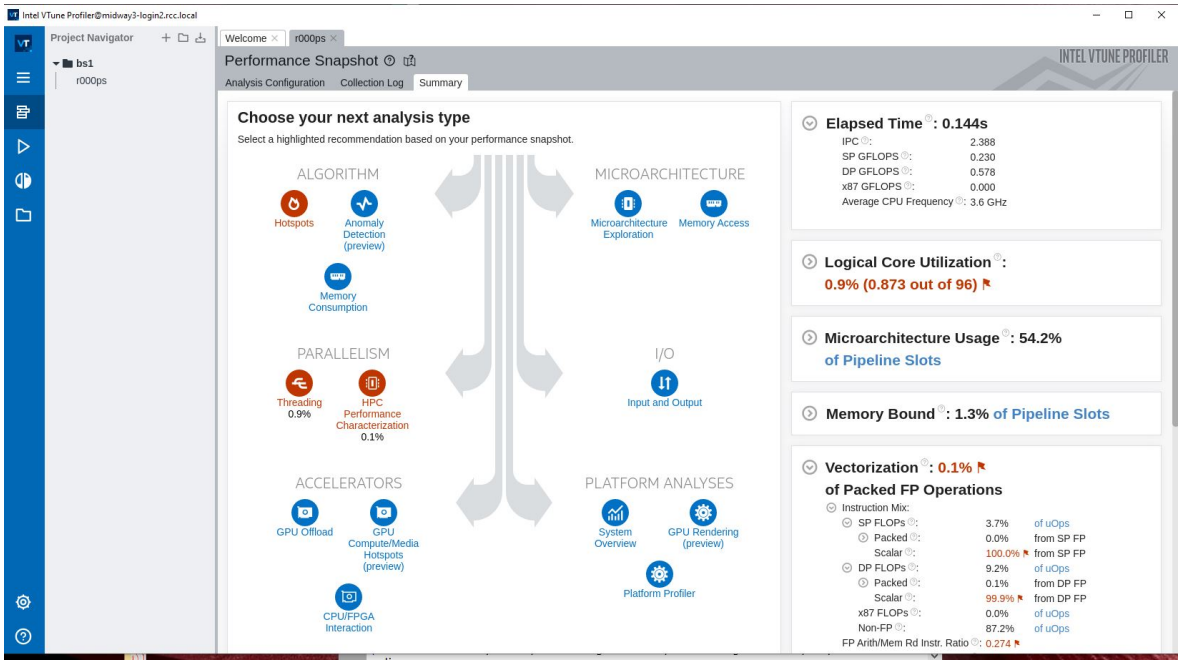
Selecting the Application

- Next, select the application to profile.
- We start with the `bs1` application:
  - This is the baseline version (no optimizations)
- Use the *Browse* button to locate the executable:



Performance Snapshot

- After launching the analysis, you will see the performance snapshot:



- This provides a high-level overview of performance.
- From here, we can explore specific performance aspects in detail.

Finding Hotspots

- Our goal is to identify performance hotspots so we use *Hotspots Analysis*
- VTune supports two sampling modes:
  1. User-Mode: focuses on application-level performance
  2. Hardware Event-Based: includes low-level hardware metrics

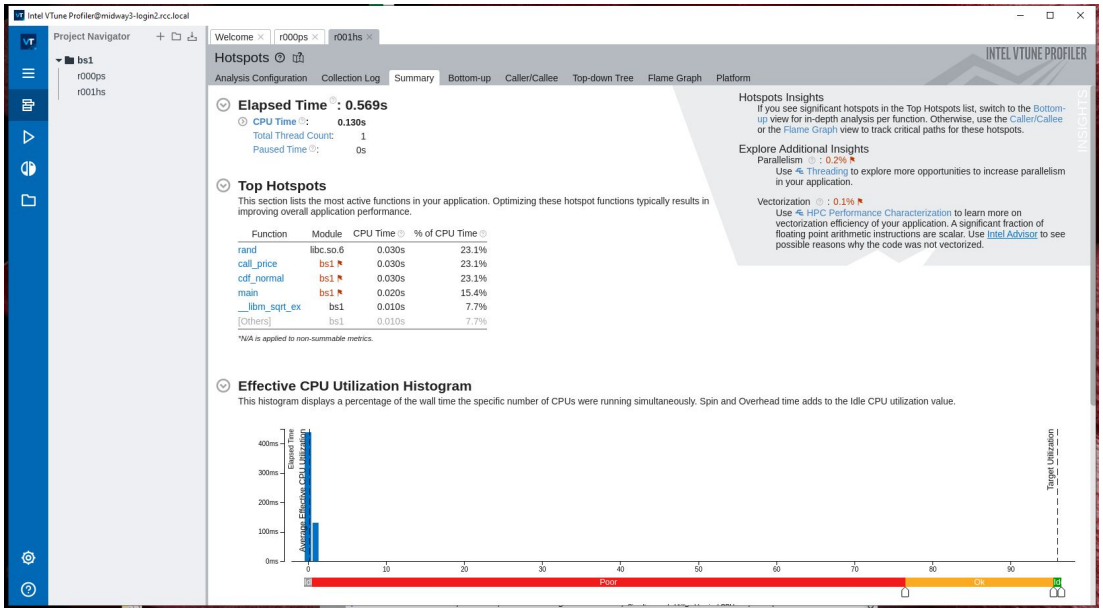
Analysis Views

- A hotspot analysis provides several views:[2]
  1. **Summary**: overall execution statistics
  2. **Bottom-Up**: hottest functions (by CPU time)
  3. **Caller/Callee**: relationships between functions
  4. **Top-Down Tree**: call hierarchy
  5. **Flame Graph**: visualization of hot call stacks
- Each view provides a different perspective on performance.

[2] Hover over column headers for descriptions

Summary Window

- Start analysis with the *Summary* window.
- Key sections:
  1. Elapsed Time
  2. Hotspots
  3. CPU Utilization Histogram

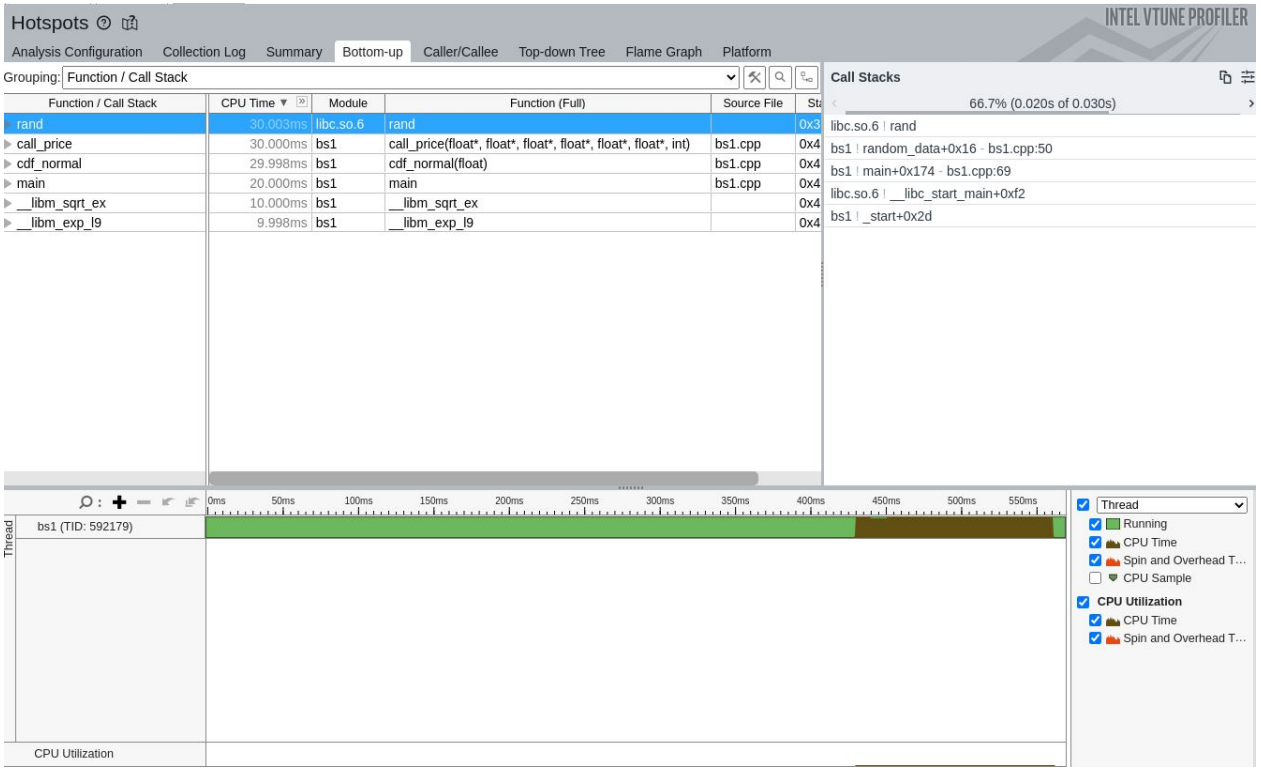


Elapsed Time vs CPU Time

- Elapsed Time:
  - Wall-clock time to run the program
  - **Primary metric we aim to reduce**
- CPU Time:
  - Single-threaded: time spent on CPU
  - Multithreaded: sum across all threads
- Note:
  - CPU time may increase with parallelism (more on this later)
  - While elapsed time decreases
- Example runtime: 569 ms (may vary by system)

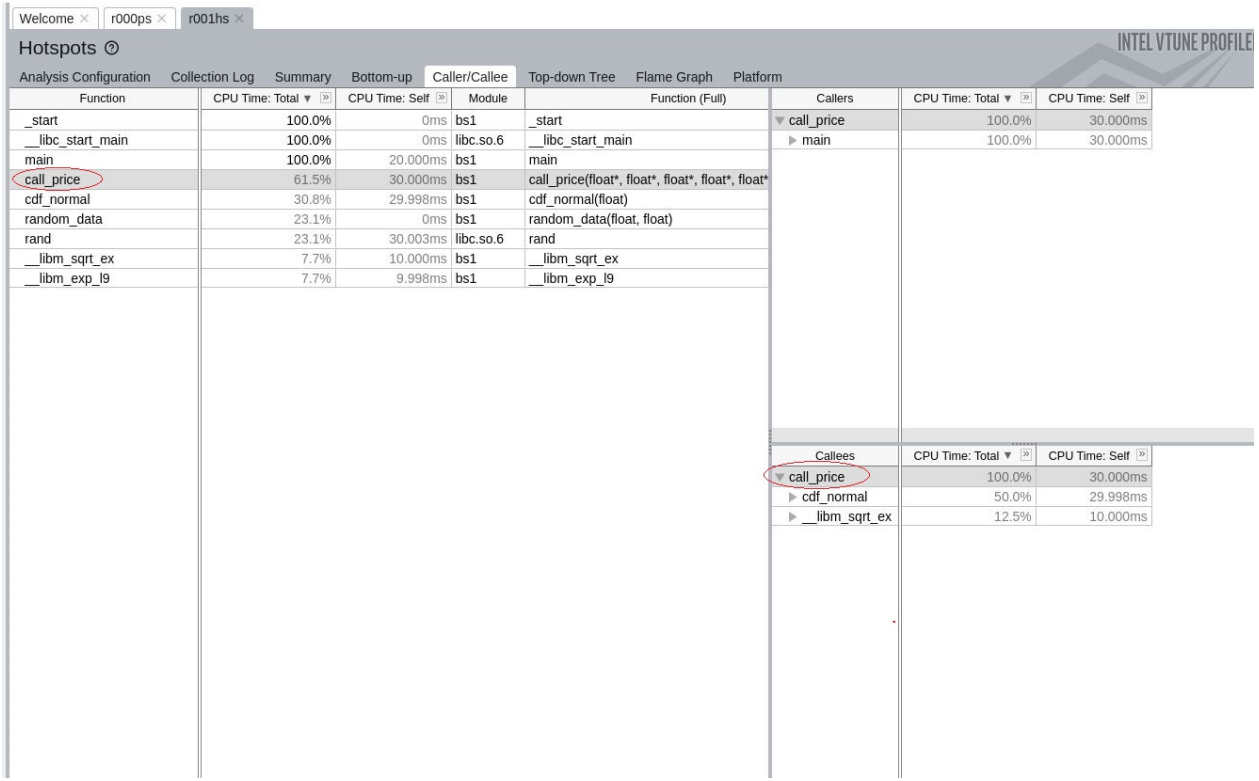
Bottom-Up View

- Shows performance from the lowest level upward.
- Functions are sorted by CPU time.
- Interpretation:
  - Top entries = most expensive functions
  - Primary candidates for optimization



Caller/Callee View

- Helps understand **function relationships**.
- Shows:
  - Who calls a function (callers)
  - What it calls (callees)
  - Time spent in each



Drilling Down into Hotspots

- Example findings:
  - 61.5% of total time spent in option pricing
- Further breakdown:
  - 50% in `cdf_normal()`
  - 12.5% in `sqrt()`
  - Within `cdf_normal()`: 12.5% in `exp()`
- Insight:
  - Performance is dominated by a small number of functions

Source-Level Analysis

- Double-click a function to inspect source code.
- VTune shows:
  - Time spent per line
  - Hot regions within functions

```
27
28  void call_price(float* S0, float *K, float *T, float* v,
29      float* r, float* C, int NumOptions)
30  {
31      for (int i = 0; i < NumOptions; ++i)
32      {
33          float d1 =                                                          7.7%   9.999ms
34              (log(S0[i] / K[i]) + (r[i] + 0.5*v[i] * v[i]) * T[i]) / (v[i] * sqrt(T[i]));   15.4%   20.000ms
35          float d2 =
36              (log(S0[i] / K[i]) + (r[i] - 0.5*v[i] * v[i]) * T[i]) / (v[i] * sqrt(T[i]));   7.7%   0ms
37
38          float nd1 = cdf_normal(d1);                                         23.1%   0ms
39          float nd2 = cdf_normal(d2);                                         7.7%   0ms
40
41          C[i] = S0[i] * nd1 - K[i] * exp(-r[i] * T[i])*nd2;
42      }
43  }
44
```

Next Step: Optimization

- We now understand:
  - Where time is spent (hotspots)
  - Which functions/lines are expensive
- Next step:
  - Apply optimizations to reduce runtime
- Optimization strategies depend on:
  - Nature of the computation
  - Identified bottlenecks

General Optimization Strategies

General Optimization Areas

- Common areas for performance improvement:
  1. Vectorization
  2. Optimize computations
  3. Use efficient libraries
  4. Improve data structures and algorithms
  5. Use appropriate language features
  6. Parallelism (multicore, GPU)
- In this course, we focus mainly on **vectorization** and **parallelism**
- Other topics are covered briefly where applicable, but a detailed discussion is not possibile due to time limitations.

1. Scalar Computation (Baseline)

- Consider a simple vector addition:

```
for (int i=0; i<N; ++i)
    c[i] = a[i] + b[i];
```

- Each iteration processes **one pair of elements**:

$$a[i] \text{ and } b[i]$$

- This is called a *scalar operation*:
  - One instruction operates on one data element (or pair)
- At the hardware level:
  - Each value is loaded into a CPU register
  - The addition is performed on these registers
- Registers are small, fast storage locations inside the CPU.

Packed Registers and Instructions

- Modern CPUs (Intel/AMD) provide:
  - Packed registers: store multiple data elements
  - Vector (SIMD) instructions: operate on all elements simultaneously
  - Giving us parallelism at the instruction level at the CPU core level.
- Example: SSE (128-bit registers)

Type	Size (bits)	Elements per Register
double	64	2
float	32	4
short	16	8
char	8	16

- Instead of processing one value at a time, we can process multiple values per instruction.

What is Vectorization?

- **Vectorization** transforms scalar code into vector code.
- Instead of one operation on one pair of values
- We perform one instruction on multiple pairs of values
- This is a form of instruction-level parallelism.
- Intuition:
  - Scalar: add 1 pair at a time
  - Vectorized: add 4, 8, or 16 pairs at once
- Result: higher throughput without increasing clock speed.

SIMD vs. SISD

- SISD (Single Instruction, Single Data)
  - Traditional scalar execution
  - One instruction operates on one data element
- SIMD (Single Instruction, Multiple Data)
  - One instruction operates on multiple data elements
  - Enabled by packed registers
- Vectorization leverages SIMD hardware.
- Modern CPUs dedicate significant hardware resources to SIMD execution.

The Power of Vectorization

- Modern CPUs support wider vector registers:
  - AVX2: 256-bit
  - AVX-512: 512-bit
- Example: 512-bit register can hold:
  - 16 floats (32-bit)
  - 8 doubles (64-bit)
- Implication: up to 8–16 operations can be performed per instruction, as shown above. Even higher throughput is possible, but we will not explore those details here in order to keep the discussion beginner-friendly. Interested students are encouraged to discuss this further during office hours.
- Vectorization can yield significant speedups, especially for:
  - Numerical computing
  - Linear algebra
  - Financial simulations
  - Example: Eigen library `https://libeigen.gitlab.io/`

Auto-Vectorization

- Modern compilers can automatically transform scalar code into vectorized code.
- This process is called **auto-vectorization**.
- The compiler:
  - Analyzes loops
  - Replaces scalar instructions with SIMD (vector) instructions
- Auto-vectorization is not guaranteed:
  - Code must satisfy certain conditions
  - Compiler must be able to prove correctness
- Key questions:
  - Was my loop vectorized?
  - If not, why not?
  - How can I help the compiler?

Auto-Vectorization: Example 1

- Consider a simple loop:

```
for (int i = 0; i < N; ++i)
    c[i] = a[i] + b[i];
```

- This loop is:
  - Regular
  - Independent across iterations
- $\Rightarrow$ **Ideal candidate for auto-vectorization**

Enabling Auto-Vectorization

- Auto-vectorization requires compiler optimizations.
- Typical behavior:
  - `-O2` or higher ⇒ vectorization enabled
  - `-O0`, `-O1` ⇒ no vectorization
- Example:

```
icc -O2 example1.cpp -o example1
```

- Question: Was the loop vectorized?

Vectorization Reports

- Compilers can generate **vectorization reports**.
- Reports tell us:
  - Which loops were vectorized
  - Which were not
  - Why
- Generate a report:

  `-qopt-report[=n]`
- Focus on vectorization:

  `-qopt-report-phase=vec`

Report Detail Levels

- Report levels:
  - n=1: Loops successfully vectorized
  - n=2: Non-vectorized loops + reasons
  - n=3: Adds dependency information
  - n=4–5: Focus on non-vectorized loops (more detail)
- Output files:
  - Linux: `.optrpt`
  - Windows: `.rep`

example1.optrpt

- Report shows:

```
LOOP BEGIN at example1.cpp(11,4)
   remark #15300: LOOP WAS VECTORIZED
LOOP END
```

Auto-Vectorization: Example 2

- Multiple loops[3]:

```
for (int row = 0; row < ROWS; ++row) {
      for (int col = 0; col < COLS; ++col) {
          data[row][col] = row + col;
      }
}

for (int row = 0; row < ROWS; ++row) {
      for (int col = 0; col < COLS; ++col) {
         sum += data[row][col] + data[col][row];
      }
}
```

[3] Not a meaningful program, for illustration only

example2.optrpt

- Report shows:

```
LOOP BEGIN at example2.cpp(13,7)
      ....
      remark #15300: LOOP WAS VECTORIZED

 LOOP BEGIN at example2.cpp(21,7)
      ...
      remark #15335: loop was not vectorized: vectorization possible but s
```

- Key observation:
  - First loop: vectorized
  - Second loop: not vectorized

Intel Advisor

- Intel Advisor is a performance analysis tool.
- Helps identify:
  - Loops that are not vectorized
  - Bottlenecks
  - Opportunities for optimization
- Provides actionable guidance:
  - What is blocking vectorization?
  - How to fix it?

Common Obstacles to Vectorization

Some Common Obstacles to Vectorization

- Data dependencies between iterations
- Function calls inside loops
- Pointer aliasing (possible memory overlap)
- Irregular control flow (branches, breaks)
- Unknown or dynamic loop bounds

Compilers must guarantee correctness:

- If the compiler is unsure $\rightarrow$ it will not vectorize

Example 3: Loop-Carried Dependencies

- Example:

```
for (int i = 0; i < N; ++i)
        if(i>3) c[i] = a[i] + b[i] + c[i-1];
```

- Problem:
  - Some iterations depends on the previous one
- $\Rightarrow$ Cannot execute iterations in parallel
- This is called a loop-carried dependency

```
LOOP BEGIN at example3.cpp(10,2)
   ...
   remark #15344: loop was not vectorized: vector dependence prevents vect
```

Example 4: Pointer Aliasing

- Code:

```
void add(float *a,float *b, float *c, float *d, float *e, int N)
{
    for (int i = 0; i < N; ++i)
        a[i] = a[i] + b[i] + c[i] + d[i] + e[i];
}
```

- Report shows:

```
LOOP BEGIN at example4.cpp(6,5) inlined into example4.cpp(16,9)
   remark #15344: loop was not vectorized: vector dependence prevents vect


LOOP BEGIN at example4.cpp(6,5)
<Multiversioned v1>
   remark #25228: Loop multiversioned for Data Dependence
   remark #15300: LOOP WAS VECTORIZED
LOOP END
```

Loop Multiversioning

- Compiler may generate:
  - A vectorized version
  - A scalar version
- Decision made at runtime:
  - Based on safety (e.g., aliasing)
- Drawback:
  - Vectorization is not guaranteed

Example 5: Non-vectorizable functions

- Code:

```
for (int i=0; i<N; ++i)
{
        a[i] = rand() % 100;
        b[i] = rand() % 100;
}

for (int i = 0; i < N; ++i)
        c[i] = a[i] + b[i];
```

- Report shows:

```
remark #15382: vectorization support: call to function rand() cannot be
```

Example 6: Complex Loops

- Code:

```
while (i<N)
{
        a[i] = a[i] * b[i];

        if (a[i] < 2) break;

        ++i;
}
```

- Report shows:

```
LOOP BEGIN at example6.cpp(10,9)
   remark #15520: loop was not vectorized: loop with multiple exits cannot
```

- More info:

https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/

developer-guide-reference/2024-2/automatic-vectorization.html

Improving Vectorization

Improving Vectorization

- Some approaches:
  1. Use vectorized libraries (e.g., SVML)
  2. Use compiler directives (pragmas)
  3. Write vectorization-friendly code

Compiler Directives

- Compilers are conservative:
  - Only vectorize when correctness is guaranteed
- We can guide the compiler using pragmas:
  - `#pragma simd` – enforce vectorization
  - `#pragma ivdep` – ignore assumed dependencies
  - `#pragma vector always` – force vectorization
- Intel specific pragmas:
  `https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2024-2/intel-specific-pragma-reference.html#GUID-C2D70038-AED3-4E15-9B1A-0D9C24D9D714`
- Use with care:
  - Incorrect usage may lead to wrong results

Writing Vectorization-Friendly Code

- Guidelines:
  1. Avoid loop-carried dependencies
  2. Use simple loop structures
  3. Avoid multiple exits (break/continue)
  4. Avoid function calls inside loops
  5. Use consistent data types
- Goal:
  - Make independence between iterations obvious

Key Takeaways

- Auto-vectorization is powerful but not automatic.
- Compilers require:
  - Clear independence between iterations
- Always:
  - Enable optimizations (`-O2`, `-O3`)
  - Check vectorization reports
- When needed:
  - Guide the compiler
  - Refactor code

2. Use Appropriate Data Types and Operations

- When we use float, we should use appropriate single precision version of the function (some examples shown below):

double	float
sqrt	sqrtf
exp	expf
log	logf
erf	erff

- Example: `http://en.cppreference.com/w/c/numeric/math/sqrt`

3. Optimizing Operations

- Code we write are translated to processor instructions.
- A processor instruction takes one or more clock cycles to complete.
- Number of clock cycles (an operation takes) in general may depends on:
  - processor type
  - operation (add, multiply, divide etc.)
  - data type (int, float, double etc.)

3. Optimizing Operations

- The code we write is ultimately translated into processor instructions.
- Each instruction takes one or more clock cycles to execute.
- The number of cycles depends on:
  - Processor type
  - Operation type (add, multiply, divide, etc.)
- Understanding these factors allows us to write more efficient code and reduce unnecessary operations.
- Even small improvements matter when operations are executed millions of times, e.g., in:
  - Model calibration
  - Risk management
  - Option pricing

Optimizing Operations: Practical Techniques

- Ways to speed up code by optimizing operations:
  1. Eliminate redundant operations
     - Example: replace $a * b + a * c$ with $a * (b + c)$
     - In Black-Scholes: use `d1` to compute `d2` when terms cancel
  2. Use cheaper operations when possible
     - Example: replace $(a + b)/2$ with $0.5 * (a + b)$
  3. Precompute repeated values
     - Example: compute `sqrt(T)` once instead of multiple times in a loop
- Small savings per operation can accumulate to significant performance gains when executed millions of times.

4. Loop Optimizations

- Loops are central to performance-critical code:
  - Express repeated computation naturally
  - Keep code concise and readable
  - Often dominate runtime in numerical applications
- Because loops run many times, even small improvements can have a large impact.
- Common loop optimization techniques:
  - Vectorization: perform multiple operations per iteration (preferred)
  - Loop unrolling: reduce loop overhead and increase instruction throughput
  - Loop-invariant code: move constant computations outside the loop

Example: Loop-Invariant Code

- Consider:

```
for (int i = 0; i < N; ++i)
{
    y[i] = x[i] * (a + b);
}
```

- Problem:
  - `(a + b)` is recomputed every iteration

- Optimized version:

```
double tmp = a + b;

for (int i = 0; i < N; ++i)
{
    y[i] = x[i] * tmp;
}
```

- Avoids redundant computation inside the loop

Loop Unrolling

- Original loop:

```
for (int i = 0; i < N; ++i)
    c[i] = a[i] + b[i];
```

- Unrolled (factor of 2):

```
for (int i = 0; i < N; i += 2)
{
    c[i]   = a[i]   + b[i];
    c[i+1] = a[i+1] + b[i+1];
}
```

- Idea:
  - Execute multiple iterations per loop iteration
- Benefits:
  - Reduces loop overhead (fewer branches and index updates)
  - Improves instruction scheduling

Choosing the Unrolling Factor

- Performance depends on the unrolling factor.
- There is no universally optimal choice:
  - Depends on hardware (CPU, cache, pipeline)
  - Depends on compiler optimizations
  - Depends on problem size
- Larger unrolling is not always better:
  - May increase code size
  - May hurt cache performance
- $\Rightarrow$ Key takeaway:
  - Profile and experiment to determine the best choice
- Note:
  - Loop unrolling is still sequential (unlike vectorization)

5. Use Better Libraries

- We cannot (and should not) write everything ourselves.
- Many high-performance libraries already exist:
  - Linear algebra (e.g., BLAS, Eigen)
  - Random number generation
  - Math/statistical functions
- Key idea:
  - Well-optimized libraries outperform custom implementations.
- Why?
  - Use vectorization and parallelism internally
  - Carefully optimized for specific hardware
  - Extensively tested and benchmarked
- $\Rightarrow$ Choose libraries carefully:
  - Performance
  - Accuracy
  - Hardware support (CPU/GPU)

6. Containers and Algorithms

- *"Bad programmers worry about the code. Good programmers worry about data structures."*
  – Linus Torvalds
- Performance depends heavily on:
  - Data structures (containers)
  - Algorithms
- Different containers have different trade-offs:
  - Access time
  - Memory layout
  - Cache efficiency
- Common C++ containers:
  1. array / std::vector
  2. std::list
  3. std::map

Memory Layout Matters

- Performance is strongly affected by memory access patterns.
- Contiguous containers (e.g., `vector`):
  - Better cache locality
  - Faster iteration
- Non-contiguous containers (e.g., `list`):
  - Poor cache performance
  - Pointer chasing
- In HPC, we generally prefer contiguous memory layouts for better performance; however, there can be cases where non-contiguous layouts may be more appropriate.

Algorithms

- Algorithms solve computational problems.
- Multiple algorithms may exist for the same task:
  - Example: sorting
    - Bubble sort, Selection sort
    - Quick sort, Merge sort
- Important:
  - Better algorithm $>$ micro-optimization
- Invest in learning Data Structures and Algorithms

7. C++ Features

- C++ provides features for writing high-performance code:
  1. References and views (avoid copies)
  2. Inlining (reduce function call overhead)
  3. Move semantics (avoid unnecessary allocations)
  4. Templates (compile-time optimization)
- Key idea:
  - Avoid unnecessary work (copies, allocations) or avoid work at runtime.

Const References

- Passing by value creates a copy:
  - Memory allocation
  - Initialization
- Copies can be expensive for large objects.
- Use `const&`:
  - Avoid copying
  - Prevent modification
- Example:

```
void foo(const std::vector<double>& v);
```

Span (C++20)

- `std::span`: a non-owning view over contiguous data.
- Works with:
  - arrays
  - std::vector
  - raw pointers
- Benefits:
  - Avoid copying data
  - Safer than raw pointers
  - Express intent clearly
- Example:

```
void foo(std::span<double> data);
```

Avoid Copies: string_view

- Extracting substrings:

```
string item = str.substr(n, m);
```

- Problem:
  - Allocates memory
  - Copies data
- Solution: `string_view`
  - Non-owning reference to string data
  - No allocation, no copy

string_view: Example

- Following code will result in new memory allocations:

```
string str =
    "009eY0 ZBZX  990101C00015000 VIX 32 FALSE";

string osi = str.substr(7, 21);
```

- No memory allocations with string_view:

```
string_view str =
     "009eY0 ZBZX  990101C00015000 VIX 32 FALSE";

string_view osi = str.substr(7, 21);
```

- See demo for details.

Move Semantics

- Creating objects can be expensive.
- Temporary objects:

  ```
  currencies_[USD] = Currency("USD", 1.0);
  ```
- Problem:
  - Temporary object is created and then discarded
- Move semantics:
  - Transfer ("move") resources instead of copying
  - Avoid unnecessary allocations

Move Semantics and emplace

- Creating and inserting objects into containers can be expensive.
- Example using `push_back`:

```
std::vector<Stock> stocks;

stocks.push_back(Stock("MSFT", 400.0));
```

  - A temporary `Stock` object is created
  - Then copied or moved into the vector
- Using `emplace_back`:

```
stocks.emplace_back("MSFT", 400.0);
```

  - Object is constructed *in-place* inside the container
  - Avoids temporary objects and unnecessary copies/moves
- Key idea:
  - Move semantics reduce copying
  - `emplace` avoids creating temporaries altogether

Inlining

- Function calls have overhead.
- Inlining replaces a function call with its body:

```
inline int max(int a, int b)
{
    return a > b ? a : b;
}
```

- Benefits:
  - Eliminates call overhead
- Notes:
  - Hint to compiler (not guaranteed)
  - Not suitable for large/complex functions

Virtual Functions: Use with Care

- Enable polymorphism and flexible design.
- Why they can be expensive:
  - The actual function to execute is determined at runtime (dynamic dispatch)
  - This requires additional work compared to direct function calls
  - Typically involves an extra level of indirection (e.g., via a lookup table)
- Performance implications:
  - Harder for the compiler to inline
  - Adds overhead inside tight loops or frequently executed code
- In performance-critical code:
  - Minimize use in hot paths when possible

Modern C++ and Performance

- Modern C++ (C++20, C++23, upcoming C++26) introduces features that:
  - Improve performance without sacrificing readability
  - Enable better compiler optimizations
  - Reduce unnecessary memory allocations and copies
- Many features are especially relevant for:
  - Vectorization
  - Cache efficiency
  - Parallel programming
- Key idea:
  - **Write high-level code that compiles to efficient low-level instructions**

Compile-Time vs Run-Time Computation

- Traditionally, most computations are performed at run time.
- Modern C++ allows certain computations to be performed at **compile time**.
- Why is this useful?
  - Eliminates redundant computations at run time
  - Reduces runtime overhead
  - Enables better compiler optimizations
- Key idea:
  - Move work from run time to compile time whenever possible
- Especially important in:
  - High-performance computing (HPC)
  - Numerical/financial applications

Example: assert vs. static_assert

- Assertions are used to verify assumptions in code.
- assert (run time)
  - Evaluated during program execution
  - Program compiles even if condition is false
  - Failure occurs at run time

```
assert(x == 10);
```

- static_assert (compile time)
  - Evaluated at compile time
  - Program fails to compile if condition is false

```
static_assert(sizeof(int) == 4, "Unexpected size");
```

- Benefits of static_assert:
  - Catch errors early (before execution)
  - No runtime cost
  - Useful for templates and generic code

constexpr: Compile-Time Computation

- constexpr enables computations at compile time.
- Can be applied to:
  - Variables
  - Functions
- Key properties:
  - Value is known at compile time
  - No runtime overhead
  - Improves performance and predictability
- Important distinction:
  - const: value cannot change
  - constexpr: value is known at compile time
- Modern C++ (C++20+):
  - Significantly expanded constexpr capabilities
  - Allows complex logic at compile time

constexpr: Example 1

- Simple compile-time function:

```
constexpr int square(int x)
{
    return x * x;
}
```

- Usage:

```
constexpr int x = 5;
constexpr int result = square(x);
```

- What happens?
  - Computation is performed at compile time
  - No runtime cost for this calculation
- If inputs are not constexpr:
  - Same function executes at run time
- Key advantage:
  - One function supports both compile-time and run-time execution

constexpr: Example 2 (Fibonacci)

- Example: Fibonacci sequence

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}$$

- Compile-time implementation:

```
constexpr int fib(int n)
{
    return (n <= 1) ? n : fib(n-1) + fib(n-2);
}
```

- Usage:

```
constexpr int f = fib(10);
```

- Notes:
  - Computed entirely at compile time
  - No runtime recursion cost
- Trade-off:
  - Compile-time recursion can increase compilation time

constexpr in Modern C++ (C++20+)

- Modern C++ greatly expands constexpr capabilities.
- Now supports:
  - Loops and conditionals
  - More complex functions
  - Parts of the standard library
- Example:

```
constexpr int sum(int n)
{
    int s = 0;
    for (int i = 1; i <= n; ++i)
        s += i;
    return s;
}
```

- Key idea:
  - Compile-time programming is becoming a core feature of C++
- Benefit for HPC:
  - Precompute constants, lookup tables, parameters

Templates and Compile-Time Programming

- Templates enable compile-time polymorphism.
- Benefits:
  - Zero runtime overhead abstraction
  - Type-safe generic programming
- Advanced uses:[4]
  - Template metaprogramming (compile-time logic)
  - Expression templates (used in numeric libraries)
- Example (high-level):

```
M4 = a * M1 + M2 * M3;
```

  - Avoids intermediate temporaries
  - Enables vectorization and optimization
- Used in libraries like:
  - Eigen (linear algebra)
  - Armadillo

[4] We won't discuss them in this course.

Compile-Time Computation: Summary

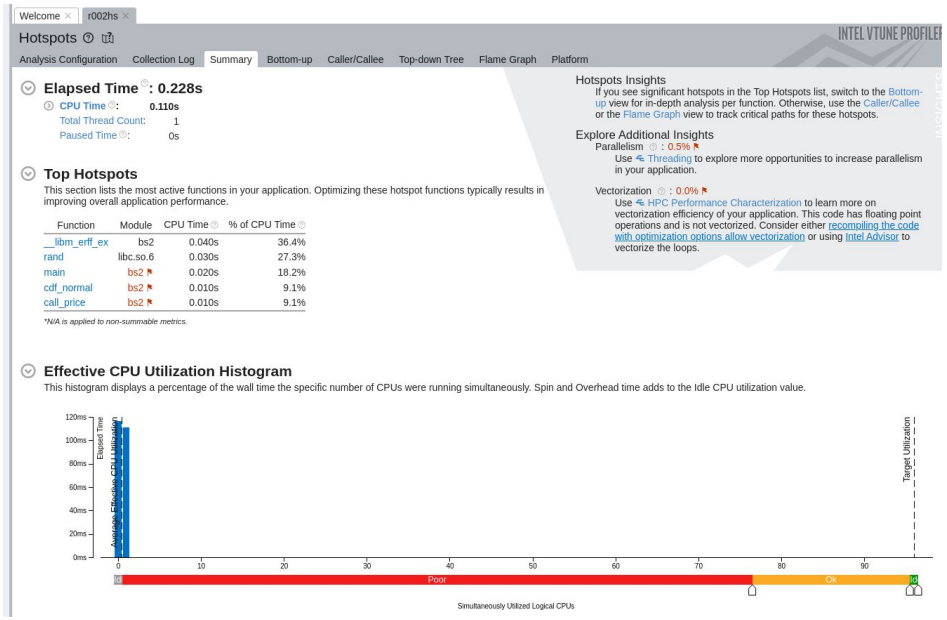
- Techniques discussed:
  - static_assert
  - constexpr
  - Templates
- Benefits:
  - Eliminate runtime work
  - Improve performance
  - Catch errors early
- Trade-offs:
  - Increased compile time
  - More complex code (sometimes)
- Key takeaway: Use compile-time computation in performance-critical code

Profile-Guided Optimization: Wrap-up

- Performance optimization is a systematic, iterative process:
  1. Identify hotspots
  2. Make targeted improvements
  3. Measure and validate changes
  4. Repeat
- We do not guess where performance issues are.
- We use profiling tools to identify performance-critical regions.
- Focus on making meaningful improvements to the identified hotspots.

Efficient Resource Utilization

- Now we're going to turn our attention to the last section in the summary view.
- Our goal is to utilize computational resources efficiently to achieve better performance.
- What kind of resources are we referring to?
- From the summary window (Effective CPU Utilization Histogram):

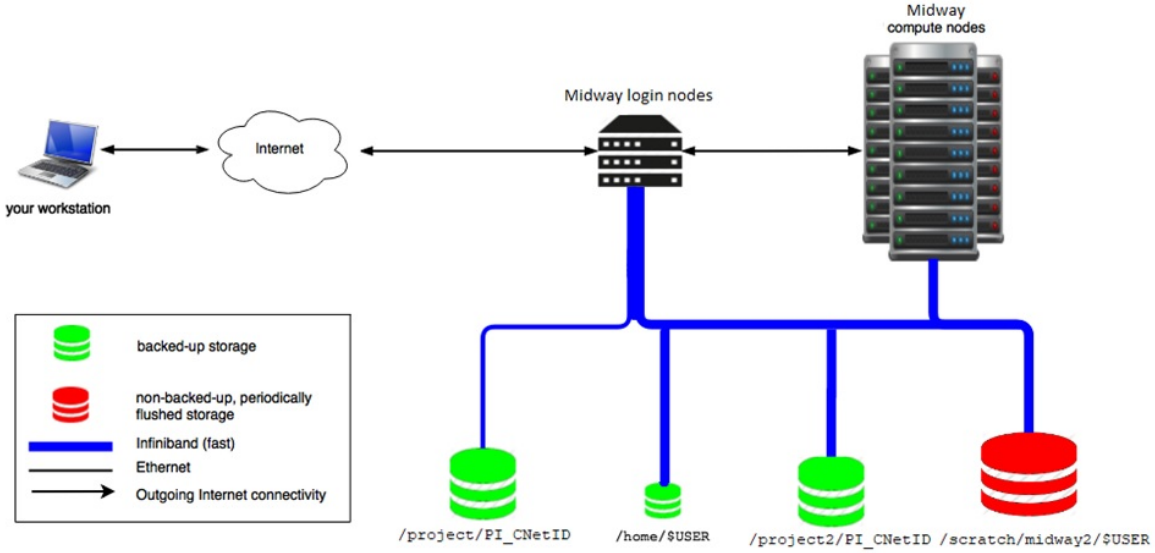


- Observation:
  - We are not utilizing CPU resources effectively
  - The application is primarily using a single core
- Even though modern systems have multiple CPU cores:
  - We are leaving significant performance potential unused
- Efficient CPU utilization is our next major topic (next week).

Appendix A: Linux and Midway (Midway3)

Midway (Midway3)

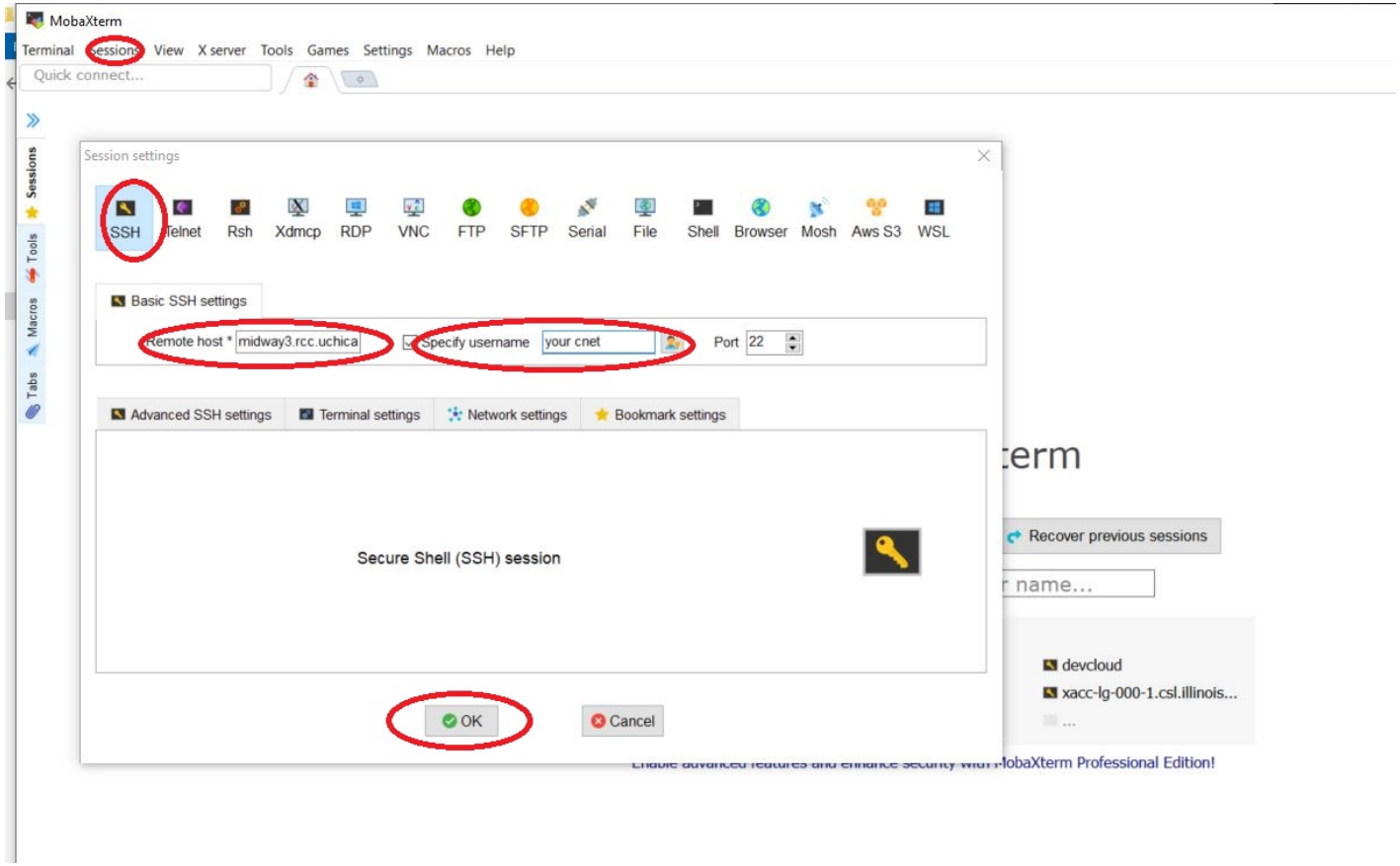
- In this course we use:
  - many software
  - many tools
  - different hardware
- We access the necessary computing resources at the Research Computing Center (RCC) at the University.
- The computing cluster we use is known as Midway (Midway3).
- Figure below shows a schematic diagram of Midway:



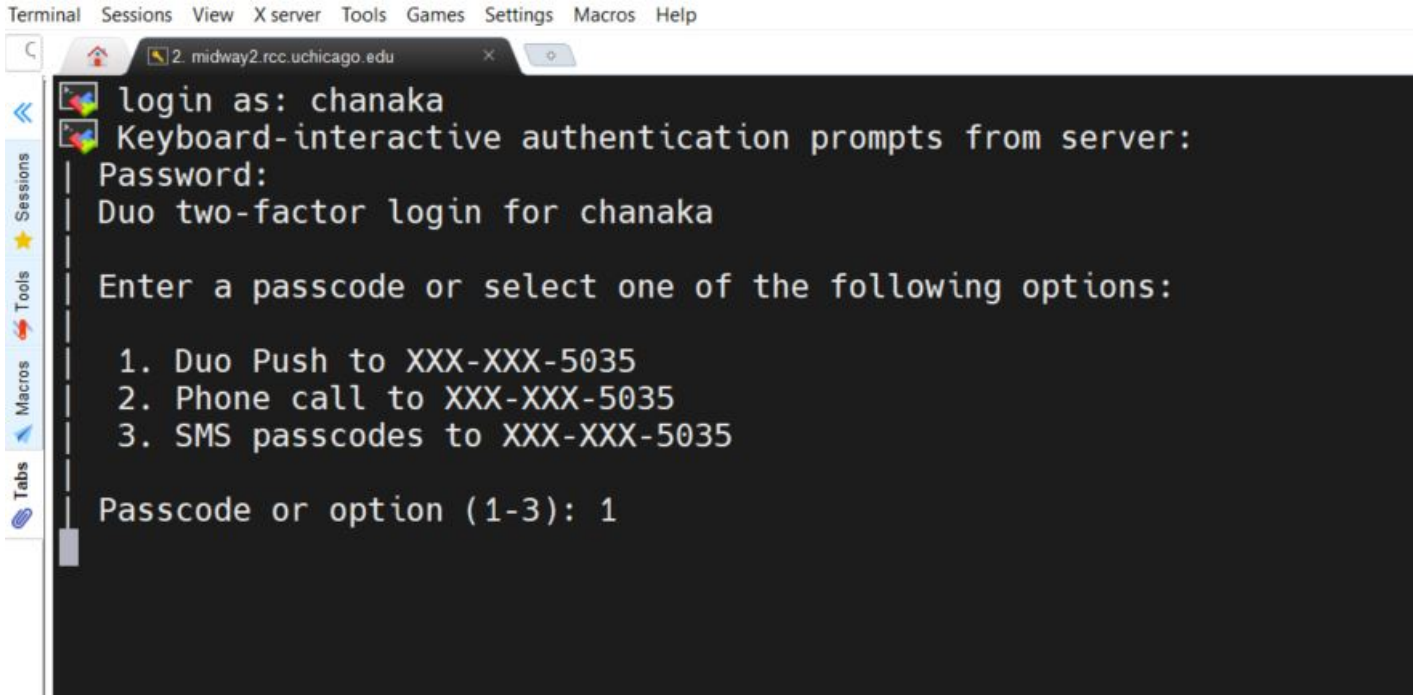
Using Midway

- MobaXterm is a good choice on Windows, but you may use any SSH client of your choice.
- To use MobaXterm:
  1. Open MobaXterm
  2. Select Session
  3. Select SSH
  4. Enter midway3.rcc.uchicago.edu as the Remote Host
  5. Click OK
  6. Use your CNet ID and password to log in

- MobaXterm:


MobaXterm
Terminal Sessions View X server Tools Games Settings Macros Help
Quick connect...
Sessions
Tools
Macros
Tabs
Session settings
SSH Telnet Rsh Xdmcp RDP VNC FTP SFTP Serial File Shell Browser Mosh Aws S3 WSL
Basic SSH settings
Remote host * midway3.rcc.uchica
Specify username your cnet
Port 22
Advanced SSH settings
Terminal settings
Network settings
Bookmark settings
Secure Shell (SSH) session
OK
Cancel
Recover previous sessions
name...
devcloud
xacc-lg-000-1.csl.illinois...
MobaXterm Professional Edition!

- Use your CNet ID and password. Midway uses 2FA for authentication.



- On Mac:
  - Open Terminal (Applications → Utilities)
  - Type the command below:

    ```
    ssh -Y YourCNet@midway3.rcc.uchicago.edu
    ```
  - Use your CNet ID and password

ThinLinc

- ThinLinc is another option, especially if you're having difficulty connecting via SSH:
  - `https://midway3.rcc.uchicago.edu`
- User guide:
  - `https://rcc-uchicago.github.io/user-guide/thinlinc/main/`

VSCode

- VSCode is a very good choice for remote development.
- However, 2FA can make the setup slightly more complex.
- **The following recommendations are based on discussions with the RCC; these are not based on my direct first-hand experience.**

RCC Recommendations - Part 1

- VS Code Remote SSH may trigger repeated password + 2FA prompts.
- This happens because multiple SSH sessions are opened during setup.
- This is a common issue when using Duo authentication.
- The following setup can make connections more consistent.
- Recommended for Mac and Linux (Windows has some limitations).

RCC Recommendations - Part 2

- Allows multiple SSH sessions to share a single authenticated connection.
- Reduces repeated password and Duo prompts.
- Add the following to `~/.ssh/config`:

```
Host midway3
    HostName midway3.rcc.uchicago.edu
    User cnetid
    ServerAliveInterval 60
    ServerAliveCountMax 3
    ControlMaster auto
    ControlPath ~/.ssh/control-%r@%h:%p
    ControlPersist 15m
```

RCC Recommendations - Part 3

- ControlPersist 15m keeps the connection alive for 15 minutes.
- You can increase this value if needed.
- Ensure the directory exists:

```
mkdir -p ~/.ssh
```

RCC Recommendations - Part 4

- WSL or Git for Windows OpenSSH works better than PuTTY or MobaXterm.
- In VS Code settings:
  - Set Remote.SSH: Use Local Server = true if reconnects are unstable
- Ensure VS Code uses the correct SSH config file:
  - remote.SSH.configFile

RCC Recommendations - Part 5

- You should see one Duo prompt per new master connection.
- While ControlPersist is active:
  - New sessions should not trigger additional prompts
- If prompts persist:
  - Increase ControlPersist (e.g., 30m)
  - Test using:

    ```
    ssh -vvv midway3
    ```

RCC Recommendations - Part 6

- Helps reduce idle disconnects during long sessions.
- If needed, increase:
  - ServerAliveInterval: 60 $\rightarrow$ 120
  - ServerAliveCountMax: 3 $\rightarrow$ 5

RCC Recommendations - Part 7

- VS Code installs a server under `~/.vscode-server`.
- If home directory quota is limited:
  - Redirect to project or scratch space

```
mkdir -p /project/[pi-folder]/[cnetid]/.vscode-server
ln -s /project/[pi-folder]/[cnetid]/.vscode-server ~/.vscoo
```

RCC Recommendations - Part 8

- For GUI-heavy workloads, ThinLinc is more reliable.
- Recommended approach:
  - Use VS Code locally
  - Use SSH or Slurm for execution on Midway

Home Directory

- Login will take you to your home directory.
- Every user has a private (your own) home directory.
- Others will not be able to read/write to your home directory.
- Home directories is backed up (on midway).
- You have a 30 GB quota (on midway).
- You can use scratch area if you need more space.

Login vs Compute Nodes

- Login nodes are used to:
  - login to midway
  - very short test runs
  - file transfers and submitting batch jobs
- Compute nodes are used to:
  - develop, debug code
  - regular program runs

Compute Nodes

- First login to Midway3
- Use sinteractive command to request a compute note:
  - Example below requests a compute node for 1 hour (using hours:minutes:seconds format).
  - Request a node for a short period at a time to increase the chances of getting a node (your request will be rejected if Midway doesn't have resources to fulfill your request).

```
sinteractive --time=1:0:0 --account=finm32950
```

- Midway3 user guide:

  `https://rcc-uchicago.github.io/user-guide/`

Software Modules

- Midway uses module software environment management system.
- We need to load the software modules before we use them.
- Shown below are some commonly used commands (more on this later):
  1. To see the list of available software
     `module avail`
  2. To see the list of currently loaded modules
     `module list`
  3. To load the Intel module
     `module load intel/2022.0`
  4. To unload the Intel module
     `module unload intel/2022.0`

- BackToNotes

Appendix B: Learning Linux

Learning Linux

- Linux uses a command-line interface.
- Use online resources (e.g., `https://ubuntu.com/tutorials/command-line-for-beginners#1-overview`) to learn basic Linux commands.
  - Read at least "Commonly Used Linux Commands", "The Linux File System", "Linux Files and File Permissions" sections.
- Software Carpentry Linux lessons: `http://swcarpentry.github.io/shell-novice/`
- Use online resources (e.g. `https://www.gnu.org/software/emacs/tour/`) to learn how to use Emacs editor.
  - Read at least "Why Emacs", "Before we get started...", "Basic editing commands", 'Help with commands" sections.